

CON EDISON CAPSTONE · SESSION 6 · AUG 11, 2026

SolarScan Verify

A generative-AI verification layer for Con Edison's rooftop solar scanner

Brief

Solar Scanner Optimization —
verify ambiguous roof
classifications

Team

Yongpeng Li · Victor Chan ·
Praewa Udomlertsakul · Kenji
Tannady · Tanapat Boontuam

Slice

The hard roofs: HVAC, skylights,
shadows

Tested

6 real NYC roofs + public-record
cross-checks

Why ambiguous roofs are the problem

6 real Manhattan roofs from public NYC orthoimagery (~0.5 m/px)



SOLAR ✓

511 W 182nd · 2024 · permit Completed 2022



NO SOLAR ✓

1086291 · no panels



HARD

1086408 · human uncertain — only Kimi agreed



HARD

1086361 · GPT-5.5 said solar ✗



NO SOLAR ✓

1086435 · no panels



HARD

511 W 182nd · 2018 · pre-install — vs 2024

🌸 Error A

False "solar" → field visit to a bare roof — truck + team wasted

🌸 Error B

False "no solar" → missed generation on a roof that has it

🔄 Today

Every ambiguous roof = manual review by a person

💡 Ideation

We asked the models to map the 4 briefs →
ranked where an agent adds most value →
the verification bottleneck: measurable,
bounded, human-in-the-loop

The prototype we actually built

Input: roof image **or** NYC address

| address → fetch roof from public orthoimagery

| + optional **context**: footprint, permit

↓

4 models in parallel, temp 0, same prompt

├→ Gemini 3.5 Flash-Lite

├→ GPT-5.5

├→ Kimi K3 (open-weights)

└→ Gemma 4-26B (open-weights)

| each returns {label, confidence, escalate}

↓

Decision rule: agree & conf ≥ .6 → accept

| disagree / low conf / record conflict → **human review**

↓

Corroboration: public permit + footprint checks

| (validated – see slide 7)

↓

Analyst decides – **final answer**

✅ Real & tested

24 model calls on 6 real NYC roofs · all parseable JSON · human labeler

🔄 Reproducible

fetch_nyc_roofs.py → evaluate → compare with human labels

🗺️ Multi-source checks

public records cross-checked before sign-off — **details on slide 7**

👤 Human in the loop

escalated cases are **never auto-accepted**

Evaluation: 6 real NYC roofs

6 roofs

Real Manhattan rooftops — public
NYC orthoimagery (~0.5 m/px)

2 human labelers

Labeled solar / no_solar / uncertain
before seeing model calls — 6/6
unanimous

4 models

Gemini · GPT-5.5 · Kimi K3 (open)
· Gemma (open)

Public records

LL24 solar permits + Building
Footprints cross-check

Design

- One address has a **Completed solar permit (2022)** — a known positive case
- Same roof imaged in **2018 and 2024** — tests imagery-date sensitivity
- Same prompt, temperature 0, structured JSON output (label · confidence · reason)

Model vs human on real roofs

Baseline case = clear roofs (1086291, 1086435, 511·2024) — all models agree with the human. Harder case = ambiguous roofs (1086361, 1086408, 511·2018) — models split.

Roof	Human	Gemini	GPT-5.5	Kimi K3	Gemma
1086291	no_solar	✓	✓	✓	✓
1086361	no_solar	✓	solar .76	✓	✓
1086408	uncertain	no_solar	no_solar	uncertain	no_solar
1086435	no_solar	✓	✓	✓	✓
511 · 2024	solar	✓ .99	✓ .98	✓ .97	✓ 1.0
511 · 2018	no_solar	✓	✓	✓	✓

🎯 Agreement with human

Kimi 6/6 · Gemini 5/6 · Gemma 5/6 ·
GPT-5.5 4/6

💡 Kimi is honest about doubt

The **only** model that said "uncertain" where
the human did (1086408)

⚠️ GPT-5.5 false positive

Confidently called "solar" (.76) on a bare
roof — the costly error class

What the two mistakes tell us

⚠️ GPT-5.5 on 1086361

- Human: no_solar · GPT-5.5: solar, conf 0.76
- 3 other models all said no_solar
- This is **Error A** — a false "solar" would send a truck to a bare roof
- Confidence 0.76 is above the 0.6 accept threshold — **the rule would have passed it**

💡 Kimi on 1086408

- Human: uncertain · Kimi: uncertain, conf 0.45
- Gemini, GPT-5.5 and Gemma all said no_solar with high confidence
- Kimi's doubt maps to the human's doubt — **open-weights honesty**
- This is the behavior an escalation layer wants: **doubt → human review**

Multi-source verification works

NYC Building Footprints (BBL + centroid)

↓

NYC **Orthos** tile (public, CC BY 4.0)

↓

4 models classify each roof

↓

Cross-check: LL24 solar permit by address

✓ Branch logic

10 scenarios × 2 models = **20/20** rule-following (conflict → escalate)

⏮ Time matters

511 W 182nd St: no_solar in 2018 (pre-install), solar in 2024 —
imagery date must match the question

🔄 Context injection

Models accept footprint/permit context and use it as corroborating evidence (branch test)

Surprising facts

🤔 Humans hesitated — only one model agreed

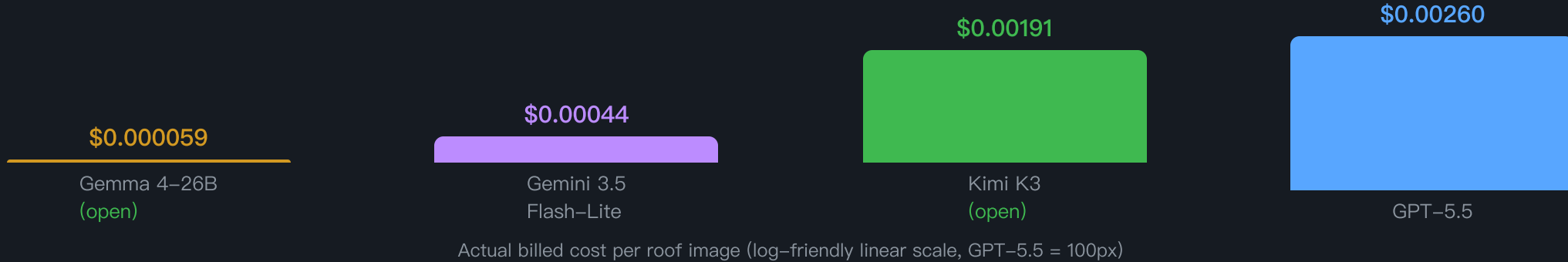
- On 1086408 the human was genuinely uncertain — and only Kimi said uncertain too
- The other three models were confidently wrong
- On 1086361, GPT-5.5 was confidently wrong in the **costly direction**

💚 Open-weights = honest about doubt

- Kimi K3 (open) matched the human 6/6 — including the uncertain case
- Gemma (open) agreed 5/6
- The closed models were more confident — and more wrong
- Kimi/Gemma are far cheaper than GPT-5.5

Model economics — real cost per roof

Measured: same rooftop image (511 W 182nd St 2024), 4 models, temp 0 — billed via OpenRouter, 2026-08-10



🏠 What one roof costs

- 4 models × 1 roof ≈ \$0.005 — half a cent for the full panel
- Open-weights 44x cheaper than GPT-5.5 (Gemma vs GPT-5.5)
- 100 roofs × 4 models ≈ \$0.50 — less than one truck visit

🔍 Why this matters

- Cost is **not the blocker** — confident wrong answers are
- The cheapest models (Kimi, Gemma) are also the **most honest about doubt** — Kimi matched the human 6/6
- So we optimize for **accuracy + honest uncertainty**, not token price

What failed → what works

✖ Failed

- **Bare-prompt classification:** inconsistent output, no confidence
- **Confident wrong answers:** GPT-5.5 called a bare roof "solar" at 0.76 — above our accept threshold
- **Costly error class is real:** false "solar" on real data, not just theory

✔ Works

- Structured output contract — 24 calls, all parseable JSON
- Clear cases separable — models agree on obvious solar/no_solar
- **Doubt signal works** — Kimi's uncertain matched the human's
- Multi-source branch — 20/20 with public records
- **Real NYC pipeline** — footprints → ortho → model → permit check

Try it live

 Local demo (Alan's Mac)

 Capstone demo (any machine)

Local = live backend on Alan's Mac (python3 scripts/demo_server.py, keys already loaded). **Capstone** = works on any machine incl. the professor's — paste an OpenRouter key for LIVE calls, or it runs pre-recorded DEMO MODE.

Address mode

Type a NYC address → we fetch the roof from public 2024 orthoimagery, 4 models classify, the decision rule fires, and the permit record cross-checks.

Try: 511 W 182nd St (solar + permit Completed 2022)

Photo mode

Drop any rooftop image → 4 models call it in parallel and we show label, confidence, reason, and a human-readable verdict.

Try: a hard roof like 1086361 (GPT-5.5 said solar, 3 said no_solar)

Roof image or address

↓

4 models in parallel (temp 0)

└→ Gemini · GPT-5.5 · Kimi K3 · Gemma

↓

Decision rule (agree & conf ≥ .6)

| accept / **escalate**

↓

Permit cross-check (LL24, public)

↓

Verdict + recommendation

Recommendation: revise, then a limited test

What the evidence supports now

- Not "stop" — the pipeline works end-to-end on real NYC roofs
- Not "deploy as-is" — one model confidently produced the costly error
- **Revise the decision rule:** add corroboration, don't trust a single confident call

What if we do nothing?

The costly error class (false "solar") appears in real data — verification is the mitigation, not the cost.



The 100-roof limited test

- With Con Edison parcels: real addresses → permit join at scale
- More labelers, more roofs, harder cases
- Also test missing/bad inputs (abstain behavior) — untested so far



What could go wrong

- Permit coverage incomplete (older/unpermitted systems)
- Imagery-date mismatch (2018 vs 2024 example)
- 2 labelers is a small panel — production needs ≥ 3
- **Missing/bad inputs** (abstain behavior) untested — part of the limited test



What we'd ask Con Edison

N roofs with parcel IDs · which context is shareable (footprints, permits, historical imagery) · a calibration set of genuinely ambiguous roofs

